

SUPPLY MANAGEMENT SYSTEM — Functional Specification Document

FSD Addendum 35 — Multi-Currency Support

Property	Value
Document ID	SMS-FSD-ADD-035
Version	1.0
Date	2026-10-07
Status	Draft
Author	System Architect
Classification	Internal — Development
Depends On	SMS-FSD-ADD-032 (Sale Inquiry & Quotation — introduced currency_code on SaleQuotations)
Architecture	.NET 8 / EF Core 8 / SQL Server 2022 / React 18 / Hangfire
Change Items	8 changes (C1-C8)
Impact	8 migrations, 10 modified entities, 3 new tables, 1 new Hangfire job, 2 new services, 6 new/modified UI panels

Table of Contents

- 1. Purpose & Scope
- 2. C1 — Currencies Lookup Table
- 3. C2 — Currency Exchange Rates with Date Range
- 4. C3 — Organization Currency Configuration (Triple Base Currency)
- 5. C4 — Business Partner Default Currency
- 6. C5 — Currency on Transaction Documents (Dual-Amount Storage)
- 7. C6 — Currency Conversion Service & Rate Locking
- 8. C7 — Exchange Rate Auto-Fetch (Hangfire Job)
- 9. C8 — Exchange Difference Accounting

10. Database Migrations
 11. API Changes
 12. UI Wireframes & Specifications
 13. Business Rules
 14. Test Scenarios
 15. Development Phases
-

1. Purpose & Scope

This addendum introduces comprehensive multi-currency support to SMS, enabling organizations to transact with customers and suppliers in any currency while maintaining separate base currencies for Sale, Purchase, and Service operations. The design follows Odoo's proven multi-currency mechanism adapted to SMS's multi-tenant architecture, with the addition of explicit date-range-based exchange rates (no implicit "most-recent-before" lookups).

1.1 Design Principles

- **Triple Base Currency** — Each tenant defines three base currencies: one for Sales, one for Purchases, and one for Services. All transaction amounts are stored in both the transaction currency AND the applicable base currency (dual-amount storage). This supports organizations that buy in USD, sell in EUR, and invoice services in GBP — each domain's financials stay clean without cross-conversion noise.
- **Explicit Date Range on Rates** — Every exchange rate has an `effective_from` and `effective_to` date. No "find the most recent row before date X" — a simple `BETWEEN` query resolves any date to exactly one rate. The current/active rate uses `effective_to = 9999-12-31`.
- **Rate Locking at Confirmation** — When a document is confirmed (SO confirmed, PO confirmed, Invoice posted), the exchange rate in effect on that date is locked onto the document header. All line conversions use this locked rate. Subsequent rate changes do not affect confirmed documents.
- **Currency Propagation** — Currency flows downstream: Customer default → Inquiry → Quotation → Sale Order → Invoice → Payment. At each stage, the currency can be overridden, but the default cascades.
- **Exchange Difference Recognition** — Realized exchange gains/losses are computed when payment is received/made at a rate different from the document's locked rate. Unrealized gains/losses are computed by a periodic revaluation job.

1.2 Changes Summary

Change ID	Title	Category
C1	Currencies Lookup Table	New table — ISO 4217 currencies with formatting metadata
C2	Currency Exchange Rates with Date Range	New table — date-ranged rates with gap/overlap prevention
C3	Organization Currency Configuration (Triple Base Currency)	Extend entity — sale_base, purchase_base, service_base on Organization
C4	Business Partner Default Currency	Extend entity — default sale/purchase currency on Business Partners
C5	Currency on Transaction Documents	Extend entities — dual-amount storage on SO, PO, Invoice, Payment
C6	Currency Conversion Service & Rate Locking	New service — ICurrencyService with rate lookup, conversion, locking
C7	Exchange Rate Auto-Fetch (Hangfire Job)	New Hangfire job — daily rate fetch from configurable external API
C8	Exchange Difference Accounting	New service — realized/unrealized gain/loss calculation and posting

1.3 Odoo Reference Mapping

Odoo Concept	SMS Equivalent	Notes
<code>res.currency</code>	<code>lookups.Currencies</code>	Same purpose — ISO 4217 master
<code>res.currency.rate</code>	<code>lookups.CurrencyRates</code>	SMS adds explicit date range (Odoo uses single-date with implicit lookup)
Company Currency	<code>sale_base_currency_id</code> / <code>purchase_base_currency_id</code> / <code>service_base_currency_id</code>	SMS splits into three — Odoo has one company currency
<code>currency_id</code> on SO/PO/Invoice	<code>currency_code</code> + <code>exchange_rate</code> + dual amounts	SMS stores converted base amount on every line
Currency Revaluation Wizard	<code>ExchangeRevaluationJob</code> (Hangfire)	Automated in SMS vs. manual wizard in Odoo

2. C1 — Currencies Lookup Table

2.1 Overview

A master table of currencies available to the organization. Seeded with ISO 4217 currencies. Each currency carries formatting metadata (symbol, decimal places, symbol position) used by the UI layer for display.

2.2 Schema — Currencies

New Table: `lookups.Currencies`

Column	Type	Nullable	Default	Description
currency_id	INT IDENTITY	NOT NULL	PK	Primary key
org_id	INT	NOT NULL	FK → tenants.Organizations	Tenant discriminator
code	NVARCHAR(3)	NOT NULL		ISO 4217 code: USD, EUR, GBP, PKR, etc.
name	NVARCHAR(60)	NOT NULL		Display name: "US Dollar", "Pakistani Rupee"
symbol	NVARCHAR(5)	NOT NULL		Display symbol: \$, €, £, Rs
decimal_places	INT	NOT NULL	2	Number of decimal places (0 for JPY/KRW, 2 for most, 3 for BHD/OMR)
rounding	DECIMAL(18,6)	NOT NULL	0.01	Smallest monetary unit (0.01, 1.00, 0.001)
symbol_position	NVARCHAR(6)	NOT NULL	'before'	Symbol placement: 'before' (\$100) or 'after' (100€)
is_active	BIT	NOT NULL	1	Soft-delete / disable toggle
display_order	INT	NOT NULL	0	Sort order in dropdowns
created_at	DATETIME2	NOT NULL	SYSUTCDATETIME()	
updated_at	DATETIME2	NOT NULL	SYSUTCDATETIME()	

Indexes:

Index	Columns	Type
UQ_Currencies_OrgCode	org_id, code	Unique
IX_Currencies_Active	org_id, is_active, display_order	Non-unique (dropdown queries)

Constraints:

Constraint	Rule
CK_Currencies_Code	LEN(code) = 3 AND code = UPPER(code)
CK_Currencies_DecimalPlaces	decimal_places BETWEEN 0 AND 3
CK_Currencies_Rounding	rounding > 0
CK_Currencies_SymbolPosition	symbol_position IN ('before', 'after')

2.3 Seed Data

 **NOTE:** On tenant creation, the system seeds these currencies. The organization admin can deactivate unused currencies and add others. PKR is included as it's a common base currency for Pakistan-based tenants, but every tenant gets the full set regardless of region — SMS is a global product.

code	name	symbol	decimal_places	rounding	symbol_position
PKR	Pakistani Rupee	Rs	2	0.01	before
USD	US Dollar	\$	2	0.01	before
EUR	Euro	€	2	0.01	before
GBP	British Pound	£	2	0.01	before
SAR	Saudi Riyal	ريال	2	0.01	before
AED	UAE Dirham	د.إ.	2	0.01	before
CNY	Chinese Yuan	¥	2	0.01	before
JPY	Japanese Yen	¥	0	1.00	before
BHD	Bahraini Dinar	BD	3	0.001	before
OMR	Omani Rial	OMR	3	0.001	before
CAD	Canadian Dollar	C\$	2	0.01	before
AUD	Australian Dollar	A\$	2	0.01	before
INR	Indian Rupee	₹	2	0.01	before
TRY	Turkish Lira	₺	2	0.01	before
MYR	Malaysian Ringgit	RM	2	0.01	before
KWD	Kuwaiti Dinar	KD	3	0.001	before
QAR	Qatari Riyal	QR	2	0.01	before
CHF	Swiss Franc	CHF	2	0.01	after

3. C2 — Currency Exchange Rates with Date Range

3.1 Overview

Exchange rates are stored with explicit date ranges (`effective_from` → `effective_to`). Every date in the continuum falls within exactly one rate record per currency — no gaps, no overlaps.

The currently active rate has `effective_to = 9999-12-31` and is closed out when a new rate is inserted.

3.2 Schema — CurrencyRates

New Table: `lookups.CurrencyRates`

Column	Type	Nullable	Default	Description
currency_rate_id	INT IDENTITY	NOT NULL	PK	Primary key
org_id	INT	NOT NULL	FK → tenants.Organizations	Tenant discriminator
currency_id	INT	NOT NULL	FK → lookups.Currencies	Which currency this rate is for
rate	DECIMAL(18,10)	NOT NULL		Units of base currency per 1 unit of foreign currency. E.g., 278.05 means 1 USD = 278.05 PKR
inverse_rate	DECIMAL(18,10)	NOT NULL		Computed: 1 / rate. For display convenience
effective_from	DATE	NOT NULL		First date this rate applies (inclusive)
effective_to	DATE	NOT NULL		Last date this rate applies (inclusive). 9999-12-31 = current/active
rate_source	NVARCHAR(30)	NOT NULL	'MANUAL'	How the rate was entered: MANUAL, API_SBP, API_ECB, API_OPENEXCHANGE, API_FOREX, SYSTEM
notes	NVARCHAR(200)	NULL		Optional note: "SBP closing rate", "Interbank mid-rate"
created_at	DATETIME2	NOT NULL	SYSUTCDATETIME()	
created_by_user_id	INT	NULL	FK → auth.Users	NULL when inserted by Hangfire job

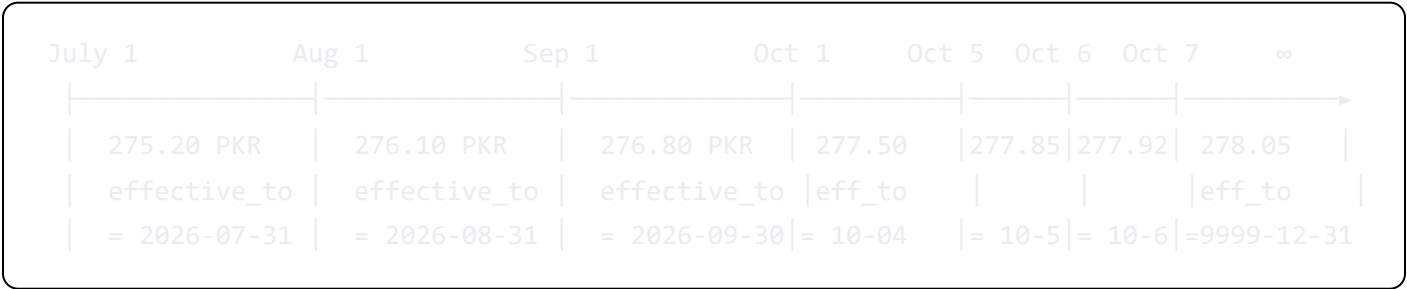
Indexes:

Index	Columns	Type
UQ_CurrencyRates_Range	org_id, currency_id, effective_from	Unique
IX_CurrencyRates_Lookup	org_id, currency_id, effective_from, effective_to	Non-unique (covering for BETWEEN queries)
IX_CurrencyRates_Active	org_id, effective_to	Non-unique, filtered (WHERE effective_to = '9999-12-31')

Constraints:

Constraint	Rule
CK_CurrencyRates_DateRange	effective_to >= effective_from
CK_CurrencyRates_RatePositive	rate > 0
CK_CurrencyRates_InversePositive	inverse_rate > 0
CK_CurrencyRates_Source	rate_source IN ('MANUAL', 'API_SBP', 'API_ECB', 'API_OPENEXCHANGE', 'API_FOREX', 'SYSTEM')

3.3 Date Range Lifecycle



Rate insertion logic (service-side, in a transaction):

```

Step 1: Find the current active rate for this currency
    WHERE org_id = @orgId AND currency_id = @currId AND effective_to = '9999-12-31'

Step 2: Close it out
    UPDATE SET effective_to = @newEffectiveFrom - 1 day

Step 3: Insert the new rate
    INSERT with effective_from = @newEffectiveFrom, effective_to = '9999-12-31'

Both steps in SERIALIZABLE transaction to prevent concurrent inserts creating overlaps.

```

3.4 Rate Lookup

```

sql

-- "What is the USD rate for October 3, 2026?"
SELECT rate, inverse_rate, effective_from, effective_to
FROM lookups.CurrencyRates
WHERE org_id = @orgId
    AND currency_id = @currencyId
    AND @transactionDate BETWEEN effective_from AND effective_to;

-- Returns exactly ONE row or ZERO rows.
-- Zero rows = no rate defined for this period → CurrencyRateNotFoundException

```

3.5 Base Currency Rate Convention

The base currency (e.g., PKR) has a permanent rate record:

```

currency_id = 1 (PKR), rate = 1.0, effective_from = '2000-01-01', effective_to = '9999-12-31'

```

This row is never closed. It ensures the conversion formula $\text{amount} \times \text{rate}$ works even when both sides are PKR (multiplying by 1.0 is a no-op).

3.6 Overlap Prevention

The unique constraint on $(\text{org_id}, \text{currency_id}, \text{effective_from})$ prevents two rates starting on the same day. The service layer additionally validates:

- **No gaps:** When inserting a rate with $\text{effective_from} = \text{Oct 5}$, the previous active row's effective_to must become Oct 4 — no day left uncovered.

- **No overlaps:** Before inserting, query for any existing rate where the new range intersects:

```
sql
WHERE org_id = @orgId AND currency_id = @currId
AND effective_from <= @newTo AND effective_to >= @newFrom
```

If rows exist beyond the current active row, reject the insert.

3.7 Manual Rate Entry for Historical Correction

An admin can edit a past rate's range or insert a rate into a gap (if one was created by a data correction). The service validates no overlaps before saving. **Locked rates on confirmed documents are NOT affected** — the rate is snapshot on the document, not referenced by FK.

4. C3 — Organization Currency Configuration (Triple Base Currency)

4.1 Overview

Each organization defines three base currencies, one per operational domain:

Domain	Column	Purpose	Example
Sale	sale_base_currency_id	All sale-side amounts (SO, Sales Invoice, Customer Payment) convert to this	EUR — company sells to European market
Purchase	purchase_base_currency_id	All purchase-side amounts (PO, Purchase Invoice, Supplier Payment) convert to this	USD — company buys from US/China suppliers
Service	service_base_currency_id	Service contracts, time-based billing, project invoicing convert to this	GBP — company's services division operates in UK

 **NOTE:** For most organizations, all three will be the same currency (e.g., all PKR). The triple-currency design is for multinational organizations or those with operationally distinct divisions. Setting all three to the same currency makes SMS behave like a single-base-currency system — no behavioral difference.

4.2 Schema — Organization Currency Settings

New Table: tenant.OrganizationCurrencySettings

Column	Type	Nullable	Default	Description
org_currency_setting_id	INT IDENTITY	NOT NULL	PK	Primary key
org_id	INT	NOT NULL	FK → tenants.Organizations	One row per org (1:1)
sale_base_currency_id	INT	NOT NULL	FK → lookups.Currencies	Base currency for all sale transactions
purchase_base_currency_id	INT	NOT NULL	FK → lookups.Currencies	Base currency for all purchase transactions
service_base_currency_id	INT	NOT NULL	FK → lookups.Currencies	Base currency for service/project transactions
exchange_rate_provider	NVARCHAR(30)	NOT NULL	'MANUAL'	Auto-fetch source: MANUAL, SBP, ECB, OPENEXCHANGE, FOREX
exchange_rate_api_key	NVARCHAR(200)	NULL		Encrypted API key for the selected provider (NULL if MANUAL or free provider)
auto_fetch_enabled	BIT	NOT NULL	0	Whether the Hangfire job should auto-fetch rates daily
auto_fetch_time	TIME	NULL	'09:00'	Local time to run the daily fetch (after market opens)
exchange_gain_account_code	NVARCHAR(20)	NULL		GL account for realized exchange

Column	Type	Nullable	Default	Description
				gains
exchange_loss_account_code	NVARCHAR(20)	NULL		GL account for realized exchange losses
unrealized_gain_account_code	NVARCHAR(20)	NULL		GL account for unrealized gains (revaluation)
unrealized_loss_account_code	NVARCHAR(20)	NULL		GL account for unrealized losses (revaluation)
created_at	DATETIME2	NOT NULL	SYSUTCDATETIME()	
updated_at	DATETIME2	NOT NULL	SYSUTCDATETIME()	

Indexes:

Index	Columns	Type
UQ_OrgCurrencySettings_Org	org_id	Unique (one row per org)

Constraints:

Constraint	Rule
CK_OrgCurrency_Provider	exchange_rate_provider IN ('MANUAL', 'SBP', 'ECB', 'OPENEXCHANGE', 'FOREX')

4.3 Initialization

On tenant creation, the system creates one row with all three base currencies set to the tenant's registration currency (typically PKR for Pakistan-registered tenants, USD for international). The admin changes these via Settings → Currency Configuration.

4.4 Base Currency Immutability Rule

 **CRITICAL:** Once a base currency has been used in a CONFIRMED transaction (any SO, PO, or Invoice posted against it), it **cannot be changed**. The UI disables the dropdown and shows a tooltip: "Cannot change — transactions exist in this base currency." This prevents retroactive misalignment of historical financials.

Workaround: Organizations that need to switch base currencies mid-year must close the period in the old currency and start a new fiscal period in the new currency (a manual process involving journal adjustments — outside the scope of this addendum).

5. C4 — Business Partner Default Currency

5.1 Overview

Each Business Partner (customer/supplier) carries a default transaction currency. When creating an Inquiry, Quotation, SO, or PO for this partner, the currency auto-fills from their default. The user can override it on any document.

5.2 Schema — BusinessPartners Extension

Modify: `suppliers.BusinessPartners`

New Column	Type	Nullable	Default	Description
default_sale_currency_id	INT	NULL	FK → lookups.Currencies	Default currency for sale documents (Inquiry, Quotation, SO, Sales Invoice). NULL = use org's sale base currency
default_purchase_currency_id	INT	NULL	FK → lookups.Currencies	Default currency for purchase documents (RFQ, PO, Purchase Invoice). NULL = use org's purchase base currency

Logic:

When creating a Sale document for Customer X:

1. If Customer X has `default_sale_currency_id` → use it
2. Else → use `org's sale_base_currency_id`

When creating a Purchase document for Supplier Y:

1. If Supplier Y has `default_purchase_currency_id` → use it
2. Else → use `org's purchase_base_currency_id`

6. C5 — Currency on Transaction Documents (Dual-Amount Storage)

6.1 Overview

Every transaction document (header + lines) carries the transaction currency and a locked exchange rate. Every monetary column on lines is stored in both currencies:

- `*_currency` — the amount in the transaction's own currency (what the customer/supplier sees)
- `*_base` — the amount converted to the applicable base currency (what accounting sees)

6.2 Dual-Amount Pattern

	Customer sees		Accountant sees
	_____		_____
<code>unit_price_currency</code>	=	AED 120.00	
		× <code>exchange_rate</code> (76.30)	
<code>unit_price_base</code>	=		PKR 9,156.00
<code>line_total_currency</code>	=	AED 6,000.00	
		× <code>exchange_rate</code> (76.30)	
<code>line_total_base</code>	=		PKR 457,800.00

6.3 Schema Modifications — Sale Side

6.3.1 SaleInquiries (demand.SaleInquiries — from ADD-032)

Modify: `demand.SaleInquiries`

New Column	Type	Nullable	Default	Description
<code>currency_id</code>	INT	NOT NULL	FK → <code>lookups.Currencies</code>	Transaction currency (defaulted from customer's <code>default_sale_currency_id</code>)

 **NOTE:** Inquiries do not lock a rate — they are pre-pricing documents. The currency here is informational, setting the stage for the Quotation.

6.3.2 SaleInquiryLines (demand.SaleInquiryLines — from ADD-032)

No monetary columns exist on inquiry lines (inquiry captures requested products/quantities, not prices). No changes needed.

6.3.3 SaleQuotations (demand.SaleQuotations — from ADD-032)

Modify: `demand.SaleQuotations`

The existing `currency_code NVARCHAR(3)` column is **replaced** with a proper FK:

Column Change	From	To	Description
REMOVE currency_code	NVARCHAR(3)	—	Drop bare string column
ADD currency_id	—	INT NOT NULL FK → lookups.Currencies	Proper FK to Currencies table
ADD exchange_rate	—	DECIMAL(18,10) NOT NULL DEFAULT 1.0	Rate locked when quotation is SENT (not at draft)
ADD base_currency_id	—	INT NOT NULL FK → lookups.Currencies	The org's sale base currency at time of creation

Modify: `demand.SaleQuotationLines`

New Column	Type	Nullable	Default	Description
unit_price_base	DECIMAL(18,4)	NOT NULL	0	$\text{unit_price} \times \text{parent exchange_rate}$
discount_amount_base	DECIMAL(18,4)	NOT NULL	0	Discount in base currency
tax_amount_base	DECIMAL(18,4)	NOT NULL	0	Tax in base currency
line_total_base	DECIMAL(18,4)	NOT NULL	0	$\text{line_total} \times \text{parent exchange_rate}$

 **NOTE:** Existing `unit_price`, `discount_percent`, `tax_amount`, `line_total` columns remain — they hold the **transaction currency** amounts. The new `*_base` columns hold the **base currency** equivalents.

6.3.4 SaleOrders (demand.SaleOrders — existing)

Modify: `demand.SaleOrders`

New Column	Type	Nullable	Default	Description
currency_id	INT	NOT NULL	FK → lookups.Currencies	Transaction currency
exchange_rate	DECIMAL(18,10)	NOT NULL	1.0	Rate locked at SO confirmation
base_currency_id	INT	NOT NULL	FK → lookups.Currencies	Org's sale base currency
subtotal_base	DECIMAL(18,4)	NOT NULL	0	Subtotal in base currency
tax_amount_base	DECIMAL(18,4)	NOT NULL	0	Tax in base currency
grand_total_base	DECIMAL(18,4)	NOT NULL	0	Grand total in base currency

Modify: `demand.SaleOrderLines`

New Column	Type	Nullable	Default	Description
unit_price_base	DECIMAL(18,4)	NOT NULL	0	$\text{unit_price} \times \text{parent exchange_rate}$
line_total_base	DECIMAL(18,4)	NOT NULL	0	$\text{line_total} \times \text{parent exchange_rate}$

6.3.5 Rate Locking Behavior — Sale Side

Document	When Rate Locks	Rate Source
Sale Inquiry	Never — no pricing	—
Sale Quotation	When status → SENT	Rate on sent_at date
Sale Order	When status → CONFIRMED	Rate on confirmation date
Sales Invoice	When status → POSTED	Rate on posting date (may differ from SO rate)

6.4 Schema Modifications — Purchase Side

6.4.1 PurchaseOrders (procurement.PurchaseOrders — existing)

Modify: `procurement.PurchaseOrders`

New Column	Type	Nullable	Default	Description
currency_id	INT	NOT NULL	FK → lookups.Currencies	Transaction currency (from supplier's default_purchase_currency_id)
exchange_rate	DECIMAL(18,10)	NOT NULL	1.0	Rate locked at PO confirmation
base_currency_id	INT	NOT NULL	FK → lookups.Currencies	Org's purchase base currency
subtotal_base	DECIMAL(18,4)	NOT NULL	0	Subtotal in base currency
tax_amount_base	DECIMAL(18,4)	NOT NULL	0	Tax in base currency
grand_total_base	DECIMAL(18,4)	NOT NULL	0	Grand total in base currency

Modify: `procurement.PurchaseOrderLines`

New Column	Type	Nullable	Default	Description
unit_price_base	DECIMAL(18,4)	NOT NULL	0	unit_price × parent exchange_rate
line_total_base	DECIMAL(18,4)	NOT NULL	0	line_total × parent exchange_rate

6.5 Schema Modifications — Finance Side

6.5.1 Sales Invoices (finance.SalesInvoices — existing or to-be-created)

Modify/Create: `finance.SalesInvoices`

New Column	Type	Nullable	Default	Description
currency_id	INT	NOT NULL	FK → lookups.Currencies	Invoice currency (carried from SO)
exchange_rate	DECIMAL(18,10)	NOT NULL	1.0	Rate locked at invoice posting
base_currency_id	INT	NOT NULL	FK → lookups.Currencies	Org's sale base currency
amount_currency	DECIMAL(18,4)	NOT NULL	0	Total in transaction currency
amount_base	DECIMAL(18,4)	NOT NULL	0	Total in base currency
exchange_rate_locked_at	DATETIME2	NULL		Timestamp when rate was locked

6.5.2 Payments (finance.Payments — existing or to-be-created)

Modify/Create: finance.Payments

New Column	Type	Nullable	Default	Description
currency_id	INT	NOT NULL	FK → lookups.Currencies	Payment currency
exchange_rate	DECIMAL(18,10)	NOT NULL	1.0	Rate on payment date
base_currency_id	INT	NOT NULL	FK → lookups.Currencies	Applicable base currency
amount_currency	DECIMAL(18,4)	NOT NULL	0	Amount paid in transaction currency
amount_base	DECIMAL(18,4)	NOT NULL	0	Amount paid in base currency
exchange_difference	DECIMAL(18,4)	NOT NULL	0	Realized gain(+) or loss(−) vs. invoice rate

6.6 Which Base Currency Is Used Where

Document Type	Base Currency Column Used	Source
Sale Inquiry	sale_base_currency_id	tenant.OrganizationCurrencySettings
Sale Quotation	sale_base_currency_id	tenant.OrganizationCurrencySettings
Sale Order	sale_base_currency_id	tenant.OrganizationCurrencySettings
Sales Invoice	sale_base_currency_id	tenant.OrganizationCurrencySettings
Customer Payment	sale_base_currency_id	tenant.OrganizationCurrencySettings
Purchase Order	purchase_base_currency_id	tenant.OrganizationCurrencySettings
Purchase Invoice	purchase_base_currency_id	tenant.OrganizationCurrencySettings
Supplier Payment	purchase_base_currency_id	tenant.OrganizationCurrencySettings
Service Contract	service_base_currency_id	tenant.OrganizationCurrencySettings
Service Invoice	service_base_currency_id	tenant.OrganizationCurrencySettings

7. C6 — Currency Conversion Service & Rate Locking

7.1 Service Interface

csharp

```
public interface ICurrencyService
{
    /// <summary>
    /// Get the exchange rate for a currency on a specific date.
    /// Uses BETWEEN effective_from AND effective_to.
    /// Throws CurrencyRateNotFoundException if no rate covers that date.
    /// </summary>
    Task<CurrencyRate> GetRateAsync(int orgId, int currencyId, DateOnly date);

    /// <summary>
    /// Convert an amount from one currency to another via the base currency.
    /// Cross-conversion: amount → base → target.
    /// </summary>
    Task<CurrencyConversionResult> ConvertAsync(
        int orgId, decimal amount,
        int fromCurrencyId, int toCurrencyId,
        DateOnly date);

    /// <summary>
    /// Convert an amount to the applicable base currency.
    /// domain determines which base currency to use (Sale/Purchase/Service).
    /// </summary>
    Task<decimal> ToBaseCurrencyAsync(
        int orgId, decimal amount,
        int sourceCurrencyId, DateOnly date,
        TransactionDomain domain);

    /// <summary>
    /// Convert an amount FROM base currency to a target currency.
    /// </summary>
    Task<decimal> FromBaseCurrencyAsync(
        int orgId, decimal amount,
        int targetCurrencyId, DateOnly date,
        TransactionDomain domain);

    /// <summary>
    /// Lock the exchange rate for a document. Returns the rate and
    /// stores nothing – the caller persists it on the document header.
    /// </summary>
    Task<decimal> LockRateAsync(int orgId, int currencyId, DateOnly date);
}
```

```

    /// <summary>
    /// Insert a new rate, closing out the previous active rate.
    /// Wrapped in SERIALIZABLE transaction.
    /// </summary>
    Task<CurrencyRate> InsertRateAsync(
        int orgId, int currencyId, decimal rate,
        DateOnly effectiveFrom, string source,
        int? userId = null, string? notes = null);

    /// <summary>
    /// Get all active rates (effective_to = 9999-12-31) for display.
    /// </summary>
    Task<IReadOnlyList<CurrencyRate>> GetActiveRatesAsync(int orgId);

    /// <summary>
    /// Get the organization's base currency ID for a given domain.
    /// </summary>
    Task<int> GetBaseCurrencyIdAsync(int orgId, TransactionDomain domain);
}

public enum TransactionDomain
{
    Sale,
    Purchase,
    Service
}

public record CurrencyConversionResult(
    decimal OriginalAmount,
    int FromCurrencyId,
    decimal ConvertedAmount,
    int ToCurrencyId,
    decimal RateUsed,
    DateOnly RateDate);

```

7.2 Rate Lookup Implementation

csharp

```

public async Task<CurrencyRate> GetRateAsync(int orgId, int currencyId, DateOnly date)
{
    var rate = await _dbContext.CurrencyRates
        .Where(r => r.OrgId == orgId
            && r.CurrencyId == currencyId
            && r.EffectiveFrom <= date
            && r.EffectiveTo >= date)
        .SingleOrDefaultAsync();

    if (rate is null)
        throw new CurrencyRateNotFoundException(orgId, currencyId, date);

    return rate;
}

```

7.3 Cross-Currency Conversion

When converting from Currency A to Currency B (neither is the base), the service converts via the base:

A → Base → B

Example: Convert 1,000 AED to EUR (base = PKR)

Step 1: 1,000 AED × 76.30 (AED→PKR rate) = 76,300 PKR

Step 2: 76,300 PKR ÷ 316.48 (EUR→PKR rate) = 241.08 EUR

7.4 Rate Locking on Document Confirmation

csharp


```
// Inside SaleOrderService.ConfirmAsync()
public async Task ConfirmAsync(long saleOrderId)
{
    var so = await GetWithLinesAsync(saleOrderId);

    // Lock the rate
    var rate = await _currencyService.LockRateAsync(
        so.OrgId, so.CurrencyId, DateOnly.FromDateTime(DateTime.UtcNow));

    so.ExchangeRate = rate;
    so.Status = SaleOrderStatus.Confirmed;

    // Recalculate all line base amounts
    foreach (var line in so.Lines)
    {
        line.UnitPriceBase = Math.Round(line.UnitPrice * rate,
            _baseCurrencyDecimalPlaces, MidpointRounding.AwayFromZero);
        line.LineTotalBase = Math.Round(line.LineTotal * rate,
            _baseCurrencyDecimalPlaces, MidpointRounding.AwayFromZero);
    }

    // Recalculate header base totals
    so.SubtotalBase = so.Lines.Sum(l => l.LineTotalBase);
    so.TaxAmountBase = Math.Round(so.TaxAmount * rate,
        _baseCurrencyDecimalPlaces, MidpointRounding.AwayFromZero);
    so.GrandTotalBase = so.SubtotalBase + so.TaxAmountBase;

    await _dbContext.SaveChangesAsync();
}
```

7.5 Same-Currency Optimization

When the transaction currency equals the applicable base currency, `exchange_rate = 1.0` and all `*_base` columns equal their `*_currency` counterparts. No rate lookup is performed.

csharp

```
if (so.CurrencyId == baseCurrencyId)
{
    so.ExchangeRate = 1.0m;
    // *_base = *_currency for all lines (no conversion needed)
}
```

8. C7 — Exchange Rate Auto-Fetch (Hangfire Job)

8.1 Overview

A recurring Hangfire job fetches exchange rates daily from the configured external API. It runs per-tenant based on each tenant's configuration.

8.2 Job Definition

Job Name: `ExchangeRateFetchJob` **Schedule:** Runs every hour. Checks each tenant's `auto_fetch_time` and `auto_fetch_enabled`. If current UTC hour matches the tenant's configured local time, fetches rates for that tenant.

8.3 Supported Providers

Provider	API	Free Tier	Base Currency	Endpoint
SBP	State Bank of Pakistan	Yes	PKR	open market rates XML
ECB	European Central Bank	Yes	EUR	Daily reference rates XML feed
OPENEXCHANGE	Open Exchange Rates	Free (hourly)	USD	/api/latest.json
FOREX	Forex API	Free (daily)	Configurable	/api/v1/latest

8.4 Job Logic

```
csharp
```

```
public class ExchangeRateFetchJob
{
    public async Task ExecuteAsync()
    {
        var tenants = await _dbContext.OrganizationCurrencySettings
            .Where(s => s.AutoFetchEnabled)
            .ToListAsync();

        foreach (var tenant in tenants)
        {
            if (!ShouldFetchNow(tenant)) continue;

            try
            {
                var provider = _providerFactory.Create(tenant.ExchangeRateProvider);
                var rates = await provider.FetchLatestRatesAsync(tenant.ExchangeRateApiKey);

                var activeCurrencies = await _dbContext.Currencies
                    .Where(c => c.OrgId == tenant.OrgId && c.IsActive)
                    .ToListAsync();

                foreach (var currency in activeCurrencies)
                {
                    if (!rates.TryGetValue(currency.Code, out var newRate))
                        continue; // Provider doesn't cover this currency

                    var today = DateOnly.FromDateTime(DateTime.UtcNow);

                    // Check if today's rate already exists
                    var existing = await _currencyService.GetRateOrDefaultAsync(
                        tenant.OrgId, currency.CurrencyId, today);

                    if (existing != null && existing.EffectiveFrom == today)
                        continue; // Already have today's rate

                    await _currencyService.InsertRateAsync(
                        tenant.OrgId, currency.CurrencyId, newRate,
                        today, $"API_{tenant.ExchangeRateProvider}");
                }

                _logger.LogInformation(
```

```

        "Fetched {Count} rates for org {OrgId} from {Provider}",
        rates.Count, tenant.OrgId, tenant.ExchangeRateProvider);
    }
    catch (Exception ex)
    {
        _logger.LogError(ex,
            "Rate fetch failed for org {OrgId}, provider {Provider}",
            tenant.OrgId, tenant.ExchangeRateProvider);
        // Continue with next tenant – don't fail the whole batch
    }
}
}
}
}

```

8.5 Provider Adapter Interface

```

csharp

public interface IExchangeRateProvider
{
    string ProviderCode { get; } // "SBP", "ECB", etc.
    Task<Dictionary<string, decimal>> FetchLatestRatesAsync(string? apiKey);
}

```

8.6 Rate Conversion for Non-Base Providers

When the provider's base differs from the org's base currency, the job converts:

```

Provider ECB returns: 1 EUR = 1.09 USD, 1 EUR = 0.86 GBP
Org's base is PKR.
ECB also returns: 1 EUR = 316.48 PKR

Converted rates stored:
    USD: 316.48 / 1.09 = 290.35 PKR ← WRONG – this is EUR/USD, not direct PKR rate

```



CORRECTION: The provider adapter normalizes all rates to "units of org base per 1 foreign unit" before returning. If the provider reports rates in EUR and the org base is PKR, the adapter does: $\text{PKR_per_foreign} = \text{PKR_per_EUR} \times \text{EUR_per_foreign}$. The adapter needs the PKR/EUR rate (which ECB publishes) to do this.

9. C8 — Exchange Difference Accounting

9.1 Realized Exchange Difference

Computed when a payment is applied against an invoice:

csharp

```
public class ExchangeDifferenceService
{
    public ExchangeDifferenceResult CalculateRealized(
        decimal invoiceAmountCurrency,
        decimal invoiceExchangeRate,
        decimal paymentExchangeRate,
        int baseCurrencyDecimalPlaces)
    {
        var invoiceAmountBase = Math.Round(
            invoiceAmountCurrency * invoiceExchangeRate,
            baseCurrencyDecimalPlaces, MidpointRounding.AwayFromZero);

        var paymentAmountBase = Math.Round(
            invoiceAmountCurrency * paymentExchangeRate,
            baseCurrencyDecimalPlaces, MidpointRounding.AwayFromZero);

        var difference = paymentAmountBase - invoiceAmountBase;

        return new ExchangeDifferenceResult
        {
            InvoiceAmountBase = invoiceAmountBase,
            PaymentAmountBase = paymentAmountBase,
            Difference = difference,
            Type = difference > 0
                ? ExchangeDifferenceType.Gain
                : difference < 0
                    ? ExchangeDifferenceType.Loss
                    : ExchangeDifferenceType.None,
            GainLossAccountCode = difference >= 0
                ? _orgSettings.ExchangeGainAccountCode
                : _orgSettings.ExchangeLossAccountCode
        };
    }
}
```

Practical example:

Invoice: AED 10,550 at rate 76.30 → PKR 804,965.00

Payment received 7 days later, rate now 76.45 → PKR 806,547.50

Realized gain: PKR 1,582.50

Journal entry:

Dr Bank/Cash PKR 806,547.50

Cr Accounts Receivable PKR 804,965.00

Cr Exchange Gain PKR 1,582.50

9.2 Unrealized Exchange Difference (Revaluation)

Hangfire Job: `ExchangeRevaluationJob`

Runs at period-end (monthly or as configured). Revalues all open (unpaid) receivables and payables at the current rate:

csharp

```

public class ExchangeRevaluationJob
{
    public async Task ExecuteAsync(int orgId, DateOnly revaluationDate)
    {
        var currentRates = await _currencyService.GetActiveRatesAsync(orgId);

        // Open receivables (unpaid sales invoices)
        var openReceivables = await _dbContext.SalesInvoices
            .Where(i => i.OrgId == orgId && i.Status == "POSTED"
                && i.AmountDue > 0
                && i.CurrencyId != i.BaseCurrencyId) // Only foreign-currency invoices
            .ToListAsync();

        foreach (var invoice in openReceivables)
        {
            var currentRate = currentRates
                .First(r => r.CurrencyId == invoice.CurrencyId).Rate;

            var revaluedBase = Math.Round(
                invoice.AmountDue * currentRate,
                _baseCurrencyDecimalPlaces);
            var originalBase = Math.Round(
                invoice.AmountDue * invoice.ExchangeRate,
                _baseCurrencyDecimalPlaces);

            var unrealizedDifference = revaluedBase - originalBase;

            if (unrealizedDifference != 0)
            {
                // Create revaluation journal entry
                await CreateRevaluationEntryAsync(
                    orgId, invoice, unrealizedDifference, revaluationDate);
            }
        }

        // Same logic for open payables (unpaid purchase invoices)...
    }
}

```

9.3 Partial Payments

When partial payment is received, the exchange difference is calculated proportionally:

Invoice: AED 10,000 at rate 76.30

Partial payment: AED 4,000 at rate 76.50

Realized difference on the partial:

Invoice base for AED 4,000: $4,000 \times 76.30 = \text{PKR } 305,200$

Payment base for AED 4,000: $4,000 \times 76.50 = \text{PKR } 306,000$

Gain: PKR 800

Remaining AED 6,000 stays at invoice rate (76.30) until next payment or revaluation.

10. Database Migrations

10.1 Migration Summary

ID	Name	Schema	Type	Description
M1	CreateCurrencies	lookups	CREATE TABLE	Currencies master with seed data
M2	CreateCurrencyRates	lookups	CREATE TABLE	Exchange rates with date range
M3	CreateOrganizationCurrencySettings	tenant	CREATE TABLE	Triple base currency config per org
M4	AddBusinessPartnerCurrencyDefaults	suppliers	ALTER TABLE	Add default_sale_currency_id, default_purchase_currency_id to BusinessPartners
M5	AddCurrencyToSaleDocuments	demand	ALTER TABLE	Add currency columns to SaleInquiries, modify SaleQuotations (replace currency_code), add to SaleOrders + lines
M6	AddCurrencyToPurchaseDocuments	procurement	ALTER TABLE	Add currency columns to PurchaseOrders + lines
M7	AddCurrencyToFinanceDocuments	finance	ALTER TABLE	Add currency columns to SalesInvoices, Payments
M8	SeedCurrenciesAndInitialRates	lookups	DATA	Seed currencies + initial PKR=1.0 rate for existing tenants

10.2 Migration SQL

M1 — CreateCurrencies

```
sql
```

```

CREATE TABLE lookups.Currencies (
    currency_id INT IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_Currencies PRIMARY KEY,
    org_id INT NOT NULL
        CONSTRAINT FK_Currencies_Org FOREIGN KEY REFERENCES tenants.Organizations(org_id)
    code NVARCHAR(3) NOT NULL,
    name NVARCHAR(60) NOT NULL,
    symbol NVARCHAR(5) NOT NULL,
    decimal_places INT NOT NULL CONSTRAINT DF_Currencies_Decimals DEFAULT 2,
    rounding DECIMAL(18,6) NOT NULL CONSTRAINT DF_Currencies_Rounding DEFAULT 0.01,
    symbol_position NVARCHAR(6) NOT NULL CONSTRAINT DF_Currencies_SymPos DEFAULT 'before'
    is_active BIT NOT NULL CONSTRAINT DF_Currencies_Active DEFAULT 1,
    display_order INT NOT NULL CONSTRAINT DF_Currencies_Order DEFAULT 0,
    created_at DATETIME2 NOT NULL CONSTRAINT DF_Currencies_Created DEFAULT SYSUTCDATETIME
    updated_at DATETIME2 NOT NULL CONSTRAINT DF_Currencies_Updated DEFAULT SYSUTCDATETIME

    CONSTRAINT UQ_Currencies_OrgCode UNIQUE (org_id, code),
    CONSTRAINT CK_Currencies_Code CHECK (LEN(code) = 3 AND code = UPPER(code)),
    CONSTRAINT CK_Currencies_DecimalPlaces CHECK (decimal_places BETWEEN 0 AND 3),
    CONSTRAINT CK_Currencies_Rounding CHECK (rounding > 0),
    CONSTRAINT CK_Currencies_SymbolPosition CHECK (symbol_position IN ('before', 'after'))
);

CREATE INDEX IX_Currencies_Active ON lookups.Currencies(org_id, is_active, display_order)

```

M2 — CreateCurrencyRates

sql

```

CREATE TABLE lookups.CurrencyRates (
    currency_rate_id INT IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_CurrencyRates PRIMARY KEY,
    org_id INT NOT NULL
        CONSTRAINT FK_CurrencyRates_Org FOREIGN KEY REFERENCES tenants.Organizations(org_id),
    currency_id INT NOT NULL
        CONSTRAINT FK_CurrencyRates_Currency FOREIGN KEY REFERENCES lookups.Currencies(currency_id),
    rate DECIMAL(18,10) NOT NULL,
    inverse_rate DECIMAL(18,10) NOT NULL,
    effective_from DATE NOT NULL,
    effective_to DATE NOT NULL,
    rate_source NVARCHAR(30) NOT NULL CONSTRAINT DF_CurrencyRates_Source DEFAULT 'MANUAL',
    notes NVARCHAR(200) NULL,
    created_at DATETIME2 NOT NULL CONSTRAINT DF_CurrencyRates_Created DEFAULT SYSUTCDATETIME(),
    created_by_user_id INT NULL
        CONSTRAINT FK_CurrencyRates_User FOREIGN KEY REFERENCES auth.Users(user_id),

    CONSTRAINT UQ_CurrencyRates_Range UNIQUE (org_id, currency_id, effective_from),
    CONSTRAINT CK_CurrencyRates_DateRange CHECK (effective_to >= effective_from),
    CONSTRAINT CK_CurrencyRates_RatePositive CHECK (rate > 0),
    CONSTRAINT CK_CurrencyRates_InversePositive CHECK (inverse_rate > 0),
    CONSTRAINT CK_CurrencyRates_Source CHECK (rate_source IN (
        'MANUAL', 'API_SBP', 'API_ECB', 'API_OPENEXCHANGE', 'API_FOREX', 'SYSTEM'))
);

CREATE INDEX IX_CurrencyRates_Lookup
    ON lookups.CurrencyRates(org_id, currency_id, effective_from, effective_to);
CREATE INDEX IX_CurrencyRates_Active
    ON lookups.CurrencyRates(org_id, effective_to)
    WHERE effective_to = '9999-12-31';

```

M3 — CreateOrganizationCurrencySettings

sql

```

CREATE TABLE tenant.OrganizationCurrencySettings (
    org_currency_setting_id INT IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_OrgCurrencySettings PRIMARY KEY,
    org_id INT NOT NULL
        CONSTRAINT FK_OrgCurrSettings_Org FOREIGN KEY REFERENCES tenants.Organizations(org_id),
    sale_base_currency_id INT NOT NULL
        CONSTRAINT FK_OrgCurrSettings_SaleBase FOREIGN KEY REFERENCES lookups.Currencies(sale_base_currency_id),
    purchase_base_currency_id INT NOT NULL
        CONSTRAINT FK_OrgCurrSettings_PurchBase FOREIGN KEY REFERENCES lookups.Currencies(purchase_base_currency_id),
    service_base_currency_id INT NOT NULL
        CONSTRAINT FK_OrgCurrSettings_ServBase FOREIGN KEY REFERENCES lookups.Currencies(service_base_currency_id),
    exchange_rate_provider NVARCHAR(30) NOT NULL CONSTRAINT DF_OrgCurrSettings_Provider DEFAULT 'MANUAL',
    exchange_rate_api_key NVARCHAR(200) NULL,
    auto_fetch_enabled BIT NOT NULL CONSTRAINT DF_OrgCurrSettings_AutoFetch DEFAULT 0,
    auto_fetch_time TIME NULL CONSTRAINT DF_OrgCurrSettings_FetchTime DEFAULT '09:00',
    exchange_gain_account_code NVARCHAR(20) NULL,
    exchange_loss_account_code NVARCHAR(20) NULL,
    unrealized_gain_account_code NVARCHAR(20) NULL,
    unrealized_loss_account_code NVARCHAR(20) NULL,
    created_at DATETIME2 NOT NULL CONSTRAINT DF_OrgCurrSettings_Created DEFAULT SYSUTCDATETIME(),
    updated_at DATETIME2 NOT NULL CONSTRAINT DF_OrgCurrSettings_Updated DEFAULT SYSUTCDATETIME(),

    CONSTRAINT UQ_OrgCurrencySettings_Org UNIQUE (org_id),
    CONSTRAINT CK_OrgCurrSettings_Provider CHECK (exchange_rate_provider IN (
        'MANUAL', 'SBP', 'ECB', 'OPENEXCHANGE', 'FOREX'))
);

```

M4 — AddBusinessPartnerCurrencyDefaults

sql

```

ALTER TABLE suppliers.BusinessPartners
    ADD default_sale_currency_id INT NULL
        CONSTRAINT FK_BusinessPartners_SaleCurrency FOREIGN KEY REFERENCES lookups.Currencies(sale_base_currency_id),

ALTER TABLE suppliers.BusinessPartners
    ADD default_purchase_currency_id INT NULL
        CONSTRAINT FK_BusinessPartners_PurchCurrency FOREIGN KEY REFERENCES lookups.Currencies(purchase_base_currency_id),

```

M5 — AddCurrencyToSaleDocuments

sql

```

-- 5a: SaleInquiries – add currency_id
ALTER TABLE demand.SaleInquiries
    ADD currency_id INT NULL
    CONSTRAINT FK_SaleInquiries_Currency FOREIGN KEY REFERENCES lookups.Currencies(cu

-- 5b: SaleQuotations – add currency_id (proper FK), exchange_rate, base_currency_id
--     Replace existing currency_code string column with FK
ALTER TABLE demand.SaleQuotations
    ADD currency_id INT NULL
    CONSTRAINT FK_SaleQuotations_Currency FOREIGN KEY REFERENCES lookups.Currencies(c

ALTER TABLE demand.SaleQuotations
    ADD exchange_rate DECIMAL(18,10) NOT NULL CONSTRAINT DF_SaleQuot_ExchRate DEFAULT 1.0

ALTER TABLE demand.SaleQuotations
    ADD base_currency_id INT NULL
    CONSTRAINT FK_SaleQuotations_BaseCurrency FOREIGN KEY REFERENCES lookups.Currenci

-- Migrate existing currency_code data to currency_id
-- (Run after M8 seeds currencies so the FK target exists)
UPDATE sq SET sq.currency_id = c.currency_id
FROM demand.SaleQuotations sq
JOIN lookups.Currencies c ON c.code = sq.currency_code AND c.org_id = sq.org_id;

-- After migration verified, drop old column
-- ALTER TABLE demand.SaleQuotations DROP COLUMN currency_code;
-- (Deferred – run manually after verification)

-- 5c: SaleQuotationLines – add base-currency columns
ALTER TABLE demand.SaleQuotationLines
    ADD unit_price_base DECIMAL(18,4) NOT NULL CONSTRAINT DF_SQLines_UPBase DEFAULT 0;
ALTER TABLE demand.SaleQuotationLines
    ADD discount_amount_base DECIMAL(18,4) NOT NULL CONSTRAINT DF_SQLines_DiscBase DEFAULT
ALTER TABLE demand.SaleQuotationLines
    ADD tax_amount_base DECIMAL(18,4) NOT NULL CONSTRAINT DF_SQLines_TaxBase DEFAULT 0;
ALTER TABLE demand.SaleQuotationLines
    ADD line_total_base DECIMAL(18,4) NOT NULL CONSTRAINT DF_SQLines_LTBase DEFAULT 0;

-- 5d: SaleOrders – add currency columns
ALTER TABLE demand.SaleOrders
    ADD currency_id INT NULL

```

```

        CONSTRAINT FK_SaleOrders_Currency FOREIGN KEY REFERENCES lookups.Currencies(curre
ALTER TABLE demand.SaleOrders
    ADD exchange_rate DECIMAL(18,10) NOT NULL CONSTRAINT DF_SaleOrders_ExchRate DEFAULT 1
ALTER TABLE demand.SaleOrders
    ADD base_currency_id INT NULL
        CONSTRAINT FK_SaleOrders_BaseCurrency FOREIGN KEY REFERENCES lookups.Currencies(c
ALTER TABLE demand.SaleOrders
    ADD subtotal_base DECIMAL(18,4) NOT NULL CONSTRAINT DF_SaleOrders_SubBase DEFAULT 0;
ALTER TABLE demand.SaleOrders
    ADD tax_amount_base DECIMAL(18,4) NOT NULL CONSTRAINT DF_SaleOrders_TaxBase DEFAULT 0
ALTER TABLE demand.SaleOrders
    ADD grand_total_base DECIMAL(18,4) NOT NULL CONSTRAINT DF_SaleOrders_TotalBase DEFAULT

-- 5e: SaleOrderLines – add base-currency columns
ALTER TABLE demand.SaleOrderLines
    ADD unit_price_base DECIMAL(18,4) NOT NULL CONSTRAINT DF_SOLines_UPBase DEFAULT 0;
ALTER TABLE demand.SaleOrderLines
    ADD line_total_base DECIMAL(18,4) NOT NULL CONSTRAINT DF_SOLines_LTBase DEFAULT 0;

```

M6 — AddCurrencyToPurchaseDocuments

sql

```
-- PurchaseOrders
ALTER TABLE procurement.PurchaseOrders
    ADD currency_id INT NULL
        CONSTRAINT FK_PurchaseOrders_Currency FOREIGN KEY REFERENCES lookups.Currencies(c
ALTER TABLE procurement.PurchaseOrders
    ADD exchange_rate DECIMAL(18,10) NOT NULL CONSTRAINT DF_PO_ExchRate DEFAULT 1.0;
ALTER TABLE procurement.PurchaseOrders
    ADD base_currency_id INT NULL
        CONSTRAINT FK_PurchaseOrders_BaseCurrency FOREIGN KEY REFERENCES lookups.Currenci
ALTER TABLE procurement.PurchaseOrders
    ADD subtotal_base DECIMAL(18,4) NOT NULL CONSTRAINT DF_PO_SubBase DEFAULT 0;
ALTER TABLE procurement.PurchaseOrders
    ADD tax_amount_base DECIMAL(18,4) NOT NULL CONSTRAINT DF_PO_TaxBase DEFAULT 0;
ALTER TABLE procurement.PurchaseOrders
    ADD grand_total_base DECIMAL(18,4) NOT NULL CONSTRAINT DF_PO_TotalBase DEFAULT 0;

-- PurchaseOrderLines
ALTER TABLE procurement.PurchaseOrderLines
    ADD unit_price_base DECIMAL(18,4) NOT NULL CONSTRAINT DF_POLines_UPBase DEFAULT 0;
ALTER TABLE procurement.PurchaseOrderLines
    ADD line_total_base DECIMAL(18,4) NOT NULL CONSTRAINT DF_POLines_LTBase DEFAULT 0;
```

M7 — AddCurrencyToFinanceDocuments

sql


```

-- SalesInvoices (if table exists)
ALTER TABLE finance.SalesInvoices
    ADD currency_id INT NULL
        CONSTRAINT FK_SalesInvoices_Currency FOREIGN KEY REFERENCES lookups.Currencies(currency_id);
ALTER TABLE finance.SalesInvoices
    ADD exchange_rate DECIMAL(18,10) NOT NULL CONSTRAINT DF_SInv_ExchRate DEFAULT 1.0;
ALTER TABLE finance.SalesInvoices
    ADD base_currency_id INT NULL
        CONSTRAINT FK_SalesInvoices_BaseCurrency FOREIGN KEY REFERENCES lookups.Currencies(currency_id);
ALTER TABLE finance.SalesInvoices
    ADD amount_currency DECIMAL(18,4) NOT NULL CONSTRAINT DF_SInv_AmtCurr DEFAULT 0;
ALTER TABLE finance.SalesInvoices
    ADD amount_base DECIMAL(18,4) NOT NULL CONSTRAINT DF_SInv_AmtBase DEFAULT 0;
ALTER TABLE finance.SalesInvoices
    ADD exchange_rate_locked_at DATETIME2 NULL;

-- Payments (if table exists)
ALTER TABLE finance.Payments
    ADD currency_id INT NULL
        CONSTRAINT FK_Payments_Currency FOREIGN KEY REFERENCES lookups.Currencies(currency_id);
ALTER TABLE finance.Payments
    ADD exchange_rate DECIMAL(18,10) NOT NULL CONSTRAINT DF_Pay_ExchRate DEFAULT 1.0;
ALTER TABLE finance.Payments
    ADD base_currency_id INT NULL
        CONSTRAINT FK_Payments_BaseCurrency FOREIGN KEY REFERENCES lookups.Currencies(currency_id);
ALTER TABLE finance.Payments
    ADD amount_currency DECIMAL(18,4) NOT NULL CONSTRAINT DF_Pay_AmtCurr DEFAULT 0;
ALTER TABLE finance.Payments
    ADD amount_base DECIMAL(18,4) NOT NULL CONSTRAINT DF_Pay_AmtBase DEFAULT 0;
ALTER TABLE finance.Payments
    ADD exchange_difference DECIMAL(18,4) NOT NULL CONSTRAINT DF_Pay_ExchDiff DEFAULT 0;

```

M8 — SeedCurrenciesAndInitialRates

sql

```

-- Seed currencies for each existing tenant
INSERT INTO lookups.Currencies (org_id, code, name, symbol, decimal_places, rounding, symbol_position)
SELECT o.org_id, v.code, v.name, v.symbol, v.decimal_places, v.rounding, v.symbol_position
FROM tenants.Organizations o
CROSS APPLY (VALUES
    ('PKR', 'Pakistani Rupee', 'Rs', 2, 0.01, 'before', 1),
    ('USD', 'US Dollar', '$', 2, 0.01, 'before', 2),
    ('EUR', 'Euro', '€', 2, 0.01, 'before', 3),
    ('GBP', 'British Pound', '£', 2, 0.01, 'before', 4),
    ('SAR', 'Saudi Riyal', '0.01', 2, 'ريال', 'before', 5),
    ('AED', 'UAE Dirham', '0.01', 2, 'د.إ.', 'before', 6),
    ('CNY', 'Chinese Yuan', '¥', 2, 0.01, 'before', 7),
    ('JPY', 'Japanese Yen', '¥', 0, 1.00, 'before', 8),
    ('BHD', 'Bahraini Dinar', 'BD', 3, 0.001, 'before', 9),
    ('OMR', 'Omani Rial', 'OMR', 3, 0.001, 'before', 10),
    ('CAD', 'Canadian Dollar', 'C$', 2, 0.01, 'before', 11),
    ('AUD', 'Australian Dollar', 'A$', 2, 0.01, 'before', 12),
    ('INR', 'Indian Rupee', '₹', 2, 0.01, 'before', 13),
    ('TRY', 'Turkish Lira', '₺', 2, 0.01, 'before', 14),
    ('MYR', 'Malaysian Ringgit', 'RM', 2, 0.01, 'before', 15),
    ('KWD', 'Kuwaiti Dinar', 'KD', 3, 0.001, 'before', 16),
    ('QAR', 'Qatari Riyal', 'QR', 2, 0.01, 'before', 17),
    ('CHF', 'Swiss Franc', 'CHF', 2, 0.01, 'after', 18)
) AS v(code, name, symbol, decimal_places, rounding, symbol_position, display_order)
WHERE NOT EXISTS (
    SELECT 1 FROM lookups.Currencies c WHERE c.org_id = o.org_id AND c.code = v.code
);

-- Seed base currency rate (1.0) for PKR for each tenant
INSERT INTO lookups.CurrencyRates (org_id, currency_id, rate, inverse_rate, effective_from, effective_to, system_flag)
SELECT c.org_id, c.currency_id, 1.0, 1.0, '2000-01-01', '9999-12-31', 'SYSTEM'
FROM lookups.Currencies c
WHERE c.code = 'PKR'
AND NOT EXISTS (
    SELECT 1 FROM lookups.CurrencyRates cr
    WHERE cr.org_id = c.org_id AND cr.currency_id = c.currency_id
);

-- Initialize OrganizationCurrencySettings for each tenant (default PKR for all three)
INSERT INTO tenant.OrganizationCurrencySettings
    (org_id, sale_base_currency_id, purchase_base_currency_id, service_base_currency_id)

```

```
SELECT o.org_id, c.currency_id, c.currency_id, c.currency_id
FROM tenants.Organizations o
JOIN lookups.Currencies c ON c.org_id = o.org_id AND c.code = 'PKR'
WHERE NOT EXISTS (
    SELECT 1 FROM tenant.OrganizationCurrencySettings s WHERE s.org_id = o.org_id
);

-- Backfill existing SaleOrders with PKR currency (for existing data)
UPDATE so SET
    so.currency_id = c.currency_id,
    so.base_currency_id = c.currency_id,
    so.exchange_rate = 1.0,
    so.subtotal_base = so.subtotal,
    so.tax_amount_base = so.tax_amount,
    so.grand_total_base = so.grand_total
FROM demand.SaleOrders so
JOIN lookups.Currencies c ON c.org_id = so.org_id AND c.code = 'PKR'
WHERE so.currency_id IS NULL;

-- Same for PurchaseOrders
UPDATE po SET
    po.currency_id = c.currency_id,
    po.base_currency_id = c.currency_id,
    po.exchange_rate = 1.0,
    po.subtotal_base = po.subtotal,
    po.tax_amount_base = po.tax_amount,
    po.grand_total_base = po.grand_total
FROM procurement.PurchaseOrders po
JOIN lookups.Currencies c ON c.org_id = po.org_id AND c.code = 'PKR'
WHERE po.currency_id IS NULL;
```

10.3 Rollback Strategy

ID Rollback SQL

M1	<code>DROP TABLE lookups.Currencies;</code> (after clearing FKs from M2, M3, M4, M5, M6, M7)
M2	<code>DROP TABLE lookups.CurrencyRates;</code>
M3	<code>DROP TABLE tenant.OrganizationCurrencySettings;</code>
M4	<code>ALTER TABLE suppliers.BusinessPartners DROP COLUMN default_sale_currency_id, default_purchase_currency_id;</code>
M5	Drop all added columns from SaleInquiries, SaleQuotations, SaleQuotationLines, SaleOrders, SaleOrderLines
M6	Drop all added columns from PurchaseOrders, PurchaseOrderLines
M7	Drop all added columns from SalesInvoices, Payments
M8	<code>DELETE FROM lookups.CurrencyRates; DELETE FROM tenant.OrganizationCurrencySettings; DELETE FROM lookups.Currencies;</code>

11. API Changes

11.1 New Endpoints

#	Method	Route	Auth Claim	Description
1	GET	<code>/api/currencies</code>	<code>currencies.read</code>	List active currencies for the org
2	POST	<code>/api/currencies</code>	<code>currencies.manage</code>	Add a new currency
3	PUT	<code>/api/currencies/{id}</code>	<code>currencies.manage</code>	Update currency (name, symbol, active)
4	GET	<code>/api/currency-rates</code>	<code>currency-rates.read</code>	List rates with date filters
5	GET	<code>/api/currency-rates/active</code>	<code>currency-rates.read</code>	Get all current active rates (effective_to = 9999-12-31)
6	GET	<code>/api/currency-rates/{currencyId}?date={date}</code>	<code>currency-rates.read</code>	Get rate for a specific currency on a specific date
7	POST	<code>/api/currency-rates</code>	<code>currency-rates.manage</code>	Insert new rate (closes previous active)
8	PUT	<code>/api/currency-rates/{id}</code>	<code>currency-rates.manage</code>	Edit a historical rate (admin)
9	GET	<code>/api/currency-rates/history/{currencyId}</code>	<code>currency-rates.read</code>	Full rate history for a currency
10	POST	<code>/api/currency-rates/fetch-now</code>	<code>currency-rates.manage</code>	Trigger immediate rate fetch from provider
11	GET	<code>/api/organization/currency-settings</code>	<code>org-settings.read</code>	Get org's currency config
12	PUT	<code>/api/organization/currency-settings</code>	<code>org-settings.manage</code>	Update org's currency config

#	Method	Route	Auth Claim	Description
13	POST	<code>/api/currency/convert</code>	<code>currencies.read</code>	Convert amount between currencies

11.2 New Permission Claims

Claim	Description
<code>currencies.read</code>	View currencies list
<code>currencies.manage</code>	Add/edit/deactivate currencies
<code>currency-rates.read</code>	View exchange rates
<code>currency-rates.manage</code>	Add/edit rates, trigger fetch
<code>org-currency-settings.manage</code>	Change base currencies, provider config

11.3 Modified Endpoints

Existing Endpoint	Change
<code>POST /api/sale-inquiries</code>	Accept <code>currency_id</code> in body; default from customer
<code>POST /api/sale-quotations</code>	Accept <code>currency_id</code> instead of <code>currency_code</code> ; add <code>exchange_rate</code> to response
<code>POST /api/sale-orders</code>	Accept <code>currency_id</code> ; response includes <code>*_base</code> amounts
<code>GET /api/sale-orders/{id}</code>	Response includes currency info, <code>exchange_rate</code> , <code>*_base</code> on all lines
<code>POST /api/purchase-orders</code>	Accept <code>currency_id</code> ; response includes <code>*_base</code> amounts
<code>GET /api/purchase-orders/{id}</code>	Response includes currency info, <code>exchange_rate</code> , <code>*_base</code> on all lines

11.4 Conversion Endpoint

POST /api/currency/convert

Body:

```
{
  "amount": 5000.00,
  "from_currency_id": 6,          // AED
  "to_currency_id": 1,           // PKR
  "date": "2026-10-07",
  "domain": "sale"               // determines which base to transit through
}
```

Response:

```
{
  "original_amount": 5000.00,
  "from_currency": { "id": 6, "code": "AED", "symbol": "ﷵ" },
  "converted_amount": 381500.00,
  "to_currency": { "id": 1, "code": "PKR", "symbol": "₹" },
  "rate_used": 76.3000,
  "rate_date": "2026-10-07",
  "effective_from": "2026-10-07",
  "effective_to": "9999-12-31"
}
```

12. UI Wireframes & Specifications

12.1 Currency Configuration (Settings → Currencies)

Settings > Currency Configuration

Base Currencies

Sale Base Currency:

[PKR - Pakistani Rupee▼]

Purchase Base Currency:

[USD - US Dollar▼]

Service Base Currency:

[PKR - Pakistani Rupee▼]

⚠ Purchase base set to USD – all PO amounts will be converted to USD for accounting.

Exchange Rate Provider

Provider:

[State Bank of Pakistan (SBP)▼]

API Key:

[(not required for SBP)]

Auto-Fetch:

[✓] Fetch rates daily at [09:00▼] PKT

[Fetch Now]

Last fetched: Oct 7, 2026 09:00 AM

Exchange Difference Accounts

Realized Gain:

[7110 - Exchange Gain]

Realized Loss:

[7120 - Exchange Loss]

Unrealized Gain:

[7130 - Unrealized Exch Gain]

Unrealized Loss:

[7140 - Unrealized Exch Loss]

[Save Settings]

12.2 Currencies List (Settings → Currencies → Manage Currencies)

Currencies							[+ Add Currency]
Order	Code	Name	Symbol	Dec.	Active	Actions	
1	PKR	Pakistani Rupee	Rs	2	✓	 	
2	USD	US Dollar	\$	2	✓		
3	EUR	Euro	€	2	✓		
4	GBP	British Pound	£	2	✓		
5	SAR	Saudi Riyal	ر.س	2	✓		
6	AED	UAE Dirham	د.ا	2	✓		
7	CNY	Chinese Yuan	¥	2	✓		
8	JPY	Japanese Yen	¥	0	✓		
9	BHD	Bahraini Dinar	BD	3	✓		
10	INR	Indian Rupee	₹	2	☐		

 = Used as base currency (cannot deactivate)

12.3 Exchange Rates Panel (Settings → Exchange Rates)

12.4 Add/Edit Rate Dialog

Add Exchange Rate

Currency:

[USD - US Dollar ▼]

Rate (PKR/USD):

[278.1200]

Inverse:

0.003596 USD/PKR

Effective From:

[2026-10-08 ]

Effective To:

☒ Current (open-ended)

☐ Fixed: []

Source:

[Manual ▼]

Notes:

[]

 The previous active rate (278.0500 from Oct 7) will be closed to Oct 7.

[Cancel]

[Save Rate]

12.5 Sale Order — Currency Display

Sale Order: SO-2026-0501

Status: CONFIRMED

Customer: Al Rashid Trading LLC

Currency: AED - UAE Dirham Exchange Rate: 76.3000 (locked Oct 7)

Base Currency: PKR

#	Product	Qty	Price	Total (AED)	Total (PKR)
1	Widget Pro	50	AED 120.00	AED 6,000.00	PKR 457,800.00
2	Gadget Lite	100	AED 45.50	AED 4,550.00	PKR 347,165.00
Subtotal				AED 10,550.00	PKR 804,965.00
Tax (5%)				AED 527.50	PKR 40,248.25
Grand Total				AED 11,077.50	PKR 845,213.25

[Toggle: Show in AED / Show in PKR]

12.6 Business Partner — Currency Defaults

Customer: Al Rashid Trading LLC

Tab: [General] [Contacts] [Currency ●] [Addresses] [Documents]

Default Sale Currency: [AED - UAE Dirham ▼]

ⓘ New Sale Inquiries, Quotations, and Orders for this customer will default to AED. Can be overridden on each document.

Default Purchase Currency: [AED - UAE Dirham ▼]
(Used when this partner is also a supplier)

13. Business Rules

Rule ID	Change	Rule	Enforcement Point
BR-C1-01	C1	Currency code must be exactly 3 uppercase ASCII letters (ISO 4217)	CurrencyValidator
BR-C1-02	C1	Currency code must be unique per org_id	DB unique constraint
BR-C1-03	C1	A currency used as any base currency cannot be deactivated	CurrencyService.DeactivateAsync
BR-C1-04	C1	A currency referenced by any transaction document cannot be deleted — only deactivated	CurrencyService.DeactivateAsync
BR-C1-05	C1	decimal_places drives all rounding: JPY (0dp) rounds to whole yen, BHD (3dp) rounds to fils	CurrencyService + UI formatters
BR-C2-01	C2	Every rate must have effective_to >= effective_from	DB CHECK constraint
BR-C2-02	C2	No two rates for the same (org_id, currency_id) may have overlapping date ranges	CurrencyRateService.InsertRateAsync — validated in SERIALIZABLE TX
BR-C2-03	C2	No gaps allowed: inserting a new rate closes the previous active rate's effective_to to (new effective_from – 1 day)	CurrencyRateService.InsertRateAsync
BR-C2-04	C2	The base currency's rate is always 1.0 with effective_from = 2000-01-01 and effective_to = 9999-12-31 — never modified	CurrencyRateService — reject changes to base currency rate

Rule ID	Change	Rule	Enforcement Point
BR-C2-05	C2	Rate lookup must return exactly 0 or 1 rows for (org, currency, date). If 0, throw CurrencyRateNotFoundException	CurrencyService.GetRateAsync
BR-C2-06	C2	rate and inverse_rate must both be > 0	DB CHECK constraint
BR-C2-07	C2	inverse_rate = 1 / rate, computed by the service before insert	CurrencyRateService
BR-C3-01	C3	Each org must have exactly one OrganizationCurrencySettings row	DB UNIQUE + tenant creation seed
BR-C3-02	C3	All three base currencies must reference active currencies belonging to the same org	OrgCurrencySettingsValidator
BR-C3-03	C3	A base currency cannot be changed once a confirmed transaction exists against it in that domain	OrgCurrencySettingsService — check for confirmed SO/PO/Invoice
BR-C3-04	C3	API key is required when provider is OPENEXCHANGE or FOREX; optional for SBP and ECB	OrgCurrencySettingsValidator
BR-C4-01	C4	Business Partner default currencies must reference active currencies in the same org	BusinessPartnerValidator
BR-C4-02	C4	If no default sale currency is set on customer, use org's sale_base_currency_id	SaleInquiryService, SaleQuotationService, SaleOrderService
BR-C4-03	C4	If no default purchase currency is set on supplier, use org's purchase_base_currency_id	PurchaseOrderService

Rule ID	Change	Rule	Enforcement Point
BR-C5-01	C5	Every transaction document header must carry currency_id, exchange_rate, and base_currency_id	Validators on SO, PO, SQ, SI, Payment
BR-C5-02	C5	exchange_rate defaults to 1.0 (same-currency) and is locked at document confirmation	*Service.ConfirmAsync
BR-C5-03	C5	All *_base amounts = corresponding *_currency amounts × exchange_rate, rounded to base currency's decimal_places	*Service.ConfirmAsync
BR-C5-04	C5	When transaction currency = base currency, exchange_rate = 1.0 and no rate lookup is performed	CurrencyService (same-currency optimization)
BR-C5-05	C5	Quotation locks rate when status → SENT; SO locks at CONFIRMED; Invoice locks at POSTED	SaleQuotationService, SaleOrderService, InvoiceService
BR-C5-06	C5	A confirmed document's exchange_rate is immutable — subsequent rate changes do not affect it	Enforced by not recalculating after confirmation
BR-C5-07	C5	Currency propagation: Customer default → Inquiry → Quotation → SO → Invoice. Each step inherits from the previous but can be overridden before confirmation	CreateFromQuotation, CreateFromSO methods
BR-C6-01	C6	Cross-currency conversion must go through the base: A → Base → B. Never directly A → B	CurrencyService.ConvertAsync
BR-C6-02	C6	Rate locking returns the rate only — caller persists it on the document. The rate is a snapshot, not a FK reference	CurrencyService.LockRateAsync

Rule ID	Change	Rule	Enforcement Point
BR-C7-01	C7	Hangfire job runs hourly; checks each tenant's auto_fetch_time and skips if not yet due	ExchangeRateFetchJob
BR-C7-02	C7	If provider API fails, log error and continue with other tenants — never fail the batch	ExchangeRateFetchJob
BR-C7-03	C7	If today's rate already exists for a currency, skip — don't overwrite manual entries	ExchangeRateFetchJob
BR-C7-04	C7	Provider adapter normalizes rates to "units of base per 1 foreign" before returning	IExchangeRateProvider implementations
BR-C8-01	C8	Realized exchange difference = (payment_amount_currency × payment_rate) – (invoice_amount_currency × invoice_rate)	ExchangeDifferenceService
BR-C8-02	C8	Positive difference = exchange gain; negative = exchange loss	ExchangeDifferenceService
BR-C8-03	C8	Partial payments calculate exchange difference proportionally on the partial amount only	ExchangeDifferenceService
BR-C8-04	C8	Unrealized revaluation revalues open receivables/payables at current rate vs. booked rate	ExchangeRevaluationJob
BR-C8-05	C8	Same-currency documents (transaction currency = base currency) never generate exchange differences	ExchangeDifferenceService — skip when exchange_rate = 1.0

14. Test Scenarios

14.1 C1 — Currencies

#	Scenario	Input	Expected Result
T-C1-01	Create valid currency	code=MXN, name=Mexican Peso, decimal=2	Currency created, appears in list
T-C1-02	Reject invalid code length	code=US	400 — "Currency code must be exactly 3 characters"
T-C1-03	Reject lowercase code	code=usd	400 — "Currency code must be uppercase"
T-C1-04	Reject duplicate code for same org	code=USD (already exists)	409 — "Currency USD already exists"
T-C1-05	Allow same code for different org	org1: USD, org2: USD	Both created — tenant isolation
T-C1-06	Deactivate unused currency	Deactivate MXN (no transactions)	is_active = 0
T-C1-07	Reject deactivation of base currency	Deactivate PKR (is sale base)	400 — "Cannot deactivate — used as sale base currency"
T-C1-08	JPY formatting	amount=1234 in JPY	Display: ¥1,234 (no decimals)
T-C1-09	BHD formatting	amount=1.567 in BHD	Display: BD 1.567 (3 decimals)
T-C1-10	Symbol position after	amount=100 in CHF	Display: 100 CHF

14.2 C2 — Exchange Rates

#	Scenario	Input	Expected Result
T-C2-01	Insert first rate	USD, rate=278.05, from=Oct 7	Created with effective_to = 9999-12-31
T-C2-02	Insert new rate (closes previous)	USD, rate=278.12, from=Oct 8	Previous row: effective_to = Oct 7. New row: from=Oct 8, to=9999-12-31
T-C2-03	Rate lookup — exact date	Lookup USD on Oct 7	Returns 278.05
T-C2-04	Rate lookup — within range	Lookup USD on Oct 3 (range Oct 1-4)	Returns 277.50
T-C2-05	Rate lookup — no rate exists	Lookup MXN on Oct 7 (no MXN rates)	CurrencyRateNotFoundException
T-C2-06	Reject overlapping rate	Insert USD from=Oct 5 when Oct 5 already covered	409 — "Rate already exists for this date"
T-C2-07	Reject rate for base currency	Try to change PKR rate from 1.0	400 — "Base currency rate cannot be modified"
T-C2-08	Reject negative rate	rate = -278.05	400 — "Rate must be positive"
T-C2-09	Inverse rate computed	Insert rate=278.05	inverse_rate = 0.003596... (computed)
T-C2-10	Concurrent rate insert	Two threads insert for same currency, same date	SERIALIZABLE TX — one succeeds, one gets 409

#	Scenario	Input	Expected Result
T-C2-11	Manual rate not overwritten by auto-fetch	Manual rate for Oct 8 exists; Hangfire tries to insert Oct 8	Hangfire skips — manual rate preserved
T-C2-12	Date range gap prevention	Insert from Oct 10 when active rate started Oct 7	Previous closes to Oct 9; new starts Oct 10 — no gap

14.3 C3 — Org Currency Config

#	Scenario	Input	Expected Result
T-C3-01	Set all three bases to same currency	sale=PKR, purchase=PKR, service=PKR	Saved. System behaves as single-currency
T-C3-02	Set purchase base to USD	purchase_base=USD	Saved. All POs convert to USD
T-C3-03	Change base with existing confirmed SO	Change sale base from PKR to EUR; confirmed SO exists	400 — "Cannot change — confirmed transactions exist"
T-C3-04	Change base with no transactions	Change sale base from PKR to EUR; no SOs	Saved
T-C3-05	Enable auto-fetch without API key (SBP)	provider=SBP, auto_fetch=true, api_key=null	Saved — SBP is free
T-C3-06	Enable auto-fetch without API key (OPENEXCHANGE)	provider=OPENEXCHANGE, auto_fetch=true, api_key=null	400 — "API key required for Open Exchange Rates"

14.4 C4 — Business Partner Defaults

#	Scenario	Input	Expected Result
T-C4-01	Customer with AED default → create inquiry	Create inquiry for customer with default_sale=AED	Inquiry.currency_id = AED
T-C4-02	Customer with no default → create inquiry	Create inquiry for customer with default_sale=NULL	Inquiry.currency_id = org's sale base (PKR)
T-C4-03	Override currency on document	Create SO with currency=EUR for customer whose default is AED	SO.currency_id = EUR (override accepted)
T-C4-04	Supplier with USD default → create PO	Create PO for supplier with default_purchase=USD	PO.currency_id = USD

14.5 C5 — Dual-Amount Storage

#	Scenario	Input	Expected Result
T- C5- 01	SO confirmed in AED (base=PKR)	SO: $50 \times \text{AED } 120 = \text{AED } 6,000$. Rate=76.30	line_total_base = PKR 457,800.00
T- C5- 02	SO confirmed in PKR (same as base)	SO: $25 \times \text{PKR } 9,200$	exchange_rate=1.0, unit_price_base=9,200 (identical)
T- C5- 03	Quotation rate locked at SENT	Quotation sent on Oct 6	exchange_rate = Oct 6 rate for that currency
T- C5- 04	SO rate locked at CONFIRMED	SO confirmed on Oct 7	exchange_rate = Oct 7 rate
T- C5- 05	Rate immutable after lock	Rate changes to 278.12 on Oct 8; SO confirmed Oct 7 at 278.05	SO.exchange_rate stays 278.05
T- C5- 06	Currency propagation SQ → SO	SQ in EUR accepted → convert to SO	SO inherits EUR and exchange_rate from SQ (or re-locks at confirmation date)
T- C5- 07	JPY rounding	SO: $10 \times \text{JPY } 1,234$. Rate=1.86	line_total_base = PKR 22,952 (rounded to 2dp for PKR). line_total = ¥12,340 (0dp)
T- C5- 08	BHD rounding	SO: $5 \times \text{BHD } 99.750$. Rate=737.47	line amounts rounded to 3dp in BHD, 2dp in PKR
T- C5- 09	Header totals match line sums	SO with 5 lines in AED	subtotal_base = SUM(line_total_base) for all lines
T- C5- 10	No rate exists at confirmation	Confirm SO in MXN; no MXN rate defined	400 — "No exchange rate found for MXN on 2026-10-07"

14.6 C6 — Currency Service

#	Scenario	Input	Expected Result
T-C6-01	Direct conversion to base	1,000 AED → PKR at rate 76.30	76,300.00 PKR
T-C6-02	Cross-currency conversion	1,000 AED → EUR via PKR	$1000 \times 76.30 / 316.48 = 241.08$ EUR
T-C6-03	Same-currency "conversion"	5,000 PKR → PKR	5,000 PKR (no lookup, rate=1.0)
T-C6-04	Conversion with historical rate	Convert on Oct 3 (rate from Oct 1-4 range)	Uses 277.50 for USD
T-C6-05	GetBaseCurrencyIdAsync for sale domain	org with sale_base=PKR, purchase_base=USD	Returns PKR id for Sale domain; USD id for Purchase domain

14.7 C7 — Auto-Fetch Job

#	Scenario	Input	Expected Result
T-C7-01	Successful fetch from SBP	auto_fetch=true, provider=SBP	Rates inserted for all active currencies with SBP coverage
T-C7-02	Provider API down	SBP returns 500	Error logged; other tenants still processed
T-C7-03	Rate already exists today	Manual rate for USD on Oct 8	Hangfire skips USD, inserts others
T-C7-04	Fetch respects tenant isolation	Org1: auto_fetch=true, Org2: false	Only Org1's rates fetched
T-C7-05	Manual trigger (Fetch Now button)	Admin clicks Fetch Now	Rates fetched immediately regardless of scheduled time

14.8 C8 — Exchange Differences

#	Scenario	Input	Expected Result
T-C8-01	Realized gain	Invoice AED 10,550 at 76.30; payment at 76.45	Gain PKR 1,582.50 → Exchange Gain account
T-C8-02	Realized loss	Invoice EUR 5,000 at 316.48; payment at 315.90	Loss PKR 2,900.00 → Exchange Loss account
T-C8-03	No difference (same rate)	Invoice and payment both at 76.30	exchange_difference = 0
T-C8-04	Same-currency invoice	Invoice in PKR, payment in PKR	No exchange calculation performed
T-C8-05	Partial payment gain	Invoice AED 10,000 at 76.30; partial AED 4,000 at 76.50	Gain on partial: PKR 800. Remaining AED 6,000 unchanged
T-C8-06	Unrealized revaluation — gain	Open AR: EUR 5,000 at 316.48; current rate 317.00	Unrealized gain: PKR 2,600
T-C8-07	Unrealized revaluation — loss	Open AP: USD 10,000 at 278.05; current rate 277.50	Unrealized loss: PKR 5,500
T-C8-08	Revaluation skips same-currency	Open AR: PKR 50,000 (base=PKR)	Skipped — no revaluation needed

15. Development Phases

15.1 Phase Summary

Phase	Changes	Tasks	Estimated Days
Phase 1 — Foundation	C1, C2	Currencies table + Rates table + Rate service + Seed	8
Phase 2 — Configuration	C3, C4	Org currency settings + BP defaults + Settings UI	5
Phase 3 — Transaction Currency	C5, C6	Dual-amount on SO/PO/SQ/SI + CurrencyService + Rate locking	10
Phase 4 — Automation & Accounting	C7, C8	Hangfire auto-fetch + Exchange difference calculation + Revaluation	7
Total			30 dev days

15.2 Phase 1 — Foundation (8 days)

Task	Description	Days
P1-01	EF Core entity + migration M1 (Currencies)	1
P1-02	EF Core entity + migration M2 (CurrencyRates)	1
P1-03	CurrencyService — GetRateAsync, InsertRateAsync with SERIALIZABLE TX	2
P1-04	Currencies CRUD API (endpoints 1-3)	1
P1-05	Currency Rates API (endpoints 4-9)	1.5
P1-06	UI — Currencies list + Add/Edit dialog	1
P1-07	UI — Exchange Rates panel + Add/Edit rate dialog	1.5
P1-08	Migration M8 — Seed data for existing tenants	0.5
P1-09	Unit tests — rate insertion, overlap prevention, gap prevention	1

15.3 Phase 2 — Configuration (5 days)

Task	Description	Days
P2-01	EF Core entity + migration M3 (OrganizationCurrencySettings)	0.5
P2-02	Migration M4 (BP currency defaults)	0.5
P2-03	OrgCurrencySettings API (endpoints 11-12)	1
P2-04	UI — Currency Configuration settings page	1.5
P2-05	UI — BP Currency tab	0.5
P2-06	Base currency immutability enforcement	0.5
P2-07	Unit tests — config CRUD, immutability	0.5

15.4 Phase 3 — Transaction Currency (10 days)

Task	Description	Days
P3-01	Migration M5 — currency columns on sale documents	1
P3-02	Migration M6 — currency columns on purchase documents	0.5
P3-03	Migration M7 — currency columns on finance documents	0.5
P3-04	CurrencyService — ToBaseCurrencyAsync, FromBaseCurrencyAsync, ConvertAsync, LockRateAsync	1.5
P3-05	SaleOrderService — integrate rate locking at ConfirmAsync	1
P3-06	PurchaseOrderService — integrate rate locking at ConfirmAsync	0.5
P3-07	SaleQuotationService — integrate rate locking at SendAsync	0.5
P3-08	Currency propagation: Customer → Inquiry → Quotation → SO	1
P3-09	UI — SO form with dual-amount display + currency selector + toggle	1.5
P3-10	UI — PO form with dual-amount display	1
P3-11	Conversion endpoint (endpoint 13)	0.5
P3-12	Integration tests — full flow Inquiry → SO → confirm with rate lock	1

15.5 Phase 4 — Automation & Accounting (7 days)

Task	Description	Days
P4-01	IXchangeRateProvider interface + SBP adapter	1
P4-02	ECB adapter	0.5
P4-03	OpenExchangeRates adapter	0.5
P4-04	ExchangeRateFetchJob (Hangfire recurring)	1
P4-05	ExchangeDifferenceService — realized gain/loss	1
P4-06	ExchangeRevaluationJob (Hangfire) — unrealized gain/loss	1
P4-07	Integration with Payment posting (realized exchange diff)	0.5
P4-08	Unit tests — exchange difference scenarios	0.5
P4-09	End-to-end test — full cycle: SO → Invoice → Payment → exchange diff	1

15.6 Services Registry

#	Service	Interface	Schema Dependency
1	CurrencyService	ICurrencyService	lookups.Currencies, lookups.CurrencyRates, tenant.OrganizationCurrencySettings
2	ExchangeDifferenceService	IXchangeDifferenceService	finance.SalesInvoices, finance.Payments
3	ExchangeRateFetchJob	(Hangfire IJob)	lookups.CurrencyRates, tenant.OrganizationCurrencySettings
4	ExchangeRevaluationJob	(Hangfire IJob)	finance.SalesInvoices, finance.PurchaseInvoices, lookups.CurrencyRates

15.7 Dependency Graph

```
C1 (Currencies table)
├─ C2 (Currency Rates) — depends on C1
│   └─ C3 (Org Currency Config) — depends on C1
│       └─ C4 (BP Defaults) — depends on C1
│           └─ C5 (Transaction Docs) — depends on C1, C2, C3
│               └─ C6 (Currency Service) — depends on C1, C2, C3
│                   └─ C7 (Auto-Fetch Job) — depends on C2, C6
│                       └─ C8 (Exchange Diff) — depends on C5, C6
```

Appendix A — Data Examples with PKR as Sale Base, USD as Purchase Base

Scenario: Company buys in USD, sells in AED

Organization Settings:

```
sale_base_currency_id      = PKR (currency_id = 1)
purchase_base_currency_id  = USD (currency_id = 2)
service_base_currency_id   = PKR (currency_id = 1)
```

Purchase Order (PO-2026-0301) — buying from US supplier:

```
Currency: USD
Exchange Rate: 1.0 (USD→USD, because purchase base = USD)
base_currency_id: USD

Line 1: 500 × $85.00 = $42,500.00
  unit_price_base: $85.00 (same – no conversion needed)
  line_total_base: $42,500.00
```

Sale Order (SO-2026-0501) — selling to Dubai customer:

```
Currency: AED
Exchange Rate: 76.30 (AED→PKR, because sale base = PKR)
base_currency_id: PKR

Line 1: 50 × AED 120.00 = AED 6,000.00
  unit_price_base: PKR 9,156.00 (120 × 76.30)
  line_total_base: PKR 457,800.00
```

Profit calculation requires cross-conversion:

Revenue (sale base = PKR): PKR 457,800.00

Cost (purchase base = USD): \$42,500.00

To compare: Convert cost to PKR using USD→PKR rate (278.05):

$\$42,500 \times 278.05 = \text{PKR } 11,817,125.00$

Gross margin in PKR: 457,800 – portion of COGS...

(Full P&L alignment is handled by the finance/reporting module)

Scenario: Single base currency (all PKR) — simplest case

Organization Settings:

sale_base_currency_id = PKR

purchase_base_currency_id = PKR

service_base_currency_id = PKR

PO in USD:

Currency: USD, Rate: 278.05

unit_price = \$85.00 → unit_price_base = PKR 23,634.25

SO in AED:

Currency: AED, Rate: 76.30

unit_price = AED 120.00 → unit_price_base = PKR 9,156.00

SO in PKR (domestic):

Currency: PKR, Rate: 1.0

unit_price = PKR 9,200.00 → unit_price_base = PKR 9,200.00

(No conversion, no rate lookup)